`fn zcdp_delta`

Michael Shoemate (`@Shoeboxam`)

August 6, 2026

Proves soundness of `fn zcdp_delta`. This proof is an adaptation of subsection 2.3 of [CKS20].

1 Bound Derivation

Definition 1.1. (Privacy Loss Random Variable). Let $M : \mathcal{X}^n \to \mathbb{Y}$ be a randomized algorithm. Let $x, x' \in \mathcal{X}^n$ be neighboring inputs. Define $f : \mathcal{Y} \to \mathbb{R}$ by $f(y) = log\left(\frac{\mathbb{P}[M(x)=y]}{\mathbb{P}[M(x')=y]}\right)$. Let $Z = f(M(x))$, the privacy loss random variable, denoted $Z \leftarrow PrivLoss(M(x)||M(x'))$.

Lemma 1.2. [CKS20] Let $\epsilon, \delta \geq 0$. Let M: $\mathcal{X}^n \to \mathcal{Y}$ be a randomized algorithm. Then M satisfies (ϵ, δ)-differential privacy if and only if

$$\delta \geq \mathop{\mathbb{E}}_{Z \leftarrow PrivLoss(M(x)||M(x'))}[max(0, 1 - e^{\epsilon - Z})] \tag{1}$$

$$\tag{2}$$

for all $x, x' \in \mathcal{X}^n$ differing on a single element.

Proof. Fix neighboring inputs $x, x' \in \mathcal{X}^n$. Let $f : \mathcal{Y} \to \mathbb{R}$ be as in 1.1. For notational simplicity, let $Y = M(x)$, $Y' = M(x')$, $Z = f(Y)$ and $Z' = -f(Y')$. This is equivalent to $Z \leftarrow PrivLoss(M(x)||M(x'))$. Our first goal is to prove that

$$\sup_{E \subset \mathcal{Y}} \mathbb{P}[Y \in E] - e^{\epsilon}\mathbb{P}[Y' \in E] = \mathbb{E}[max\{0, 1 - e^{\epsilon - Z}\}]. \tag{3}$$

For any $E \subset \mathcal{Y}$, we have

$$\mathbb{P}[Y' \in E] = \mathbb{E}[\mathbb{I}[Y' \in E]] = \mathbb{E}[\mathbb{I}[Y \in E]e^{-f(Y)}]. \tag{4}$$

This is because $e^{-f(y)} = \frac{\mathbb{P}[Y'=y]}{\mathbb{P}[Y=y]}$.

Thus, for all $E \subset \mathcal{Y}$, we have

$$\mathbb{P}[Y \in E] - e^{\epsilon}\mathbb{P}[Y' \in E] = \mathbb{E}\left[\mathbb{I}[Y \in E](1 - e^{\epsilon - f(Y)})\right] \tag{5}$$

Now it is easy to identify the worst event as $E = \{y \in \mathcal{Y} : 1 - e^{\epsilon - f(y)} > 0\}$. Thus

$$\sup_{E \subset Y} \mathbb{P}[Y \in E] - e^{\epsilon}\mathbb{P}[Y' \in E] = \mathbb{E}\left[\mathbb{I}[1 - e^{\epsilon - f(Y)} > 0](1 - e^{\epsilon - f(Y)})\right] = \mathbb{E}[max\{0, 1 - e^{\epsilon - Z}\}] \tag{6}$$

□

Theorem 1.3. [CKS20] Let $M : \mathcal{X}^n \to \mathcal{Y}$ be a randomized algorithm. Let $\alpha \in (1, \infty)$ and $\epsilon \geq 0$. Suppose $D_\alpha(M(x)||M(x')) \leq \tau$ for all $x, x' \in \mathcal{X}^n$ differing in a single entry.[1] Then M is (ϵ, δ)-differentially private for

$$\delta = \frac{e^{(\alpha-1)(\tau-\epsilon)}}{\alpha - 1}\left(1 - \frac{1}{\alpha}\right)^\alpha \tag{7}$$

Proof. Fix neighboring $x, x' \in \mathcal{X}^n$ and let $Z \leftarrow PrivLoss(M(x)||M(x'))$. We have

$$\mathbb{E}[e^{(\alpha-1)Z}] = e^{(\alpha-1)D_\alpha(M(x)||M(x'))} \leq e^{(\alpha-1)\tau} \tag{8}$$

By 1.2, our goal is to prove that $\delta \geq \mathbb{E}[max\{0, 1 - e^{\epsilon - Z}\}]$. Our approach is to pick $c > 0$ such that $max\{0, 1 - e^{\epsilon-Z}\} \leq ce^{(\alpha-1)z}$ for all $z \in \mathbb{R}$. Then

$$\mathbb{E}[max\{0, 1 - e^{\epsilon-Z}\}] \leq \mathbb{E}[ce^{(\alpha-1)z}] \leq ce^{(\alpha-1)\tau}. \tag{9}$$

We identify the smallest possible value of c:

$$c = \sup_{z\in\mathbb{R}} \frac{max\{0, 1 - e^{\epsilon-z}\}}{e^{(\alpha-1)z}} = \sup_{z\in\mathbb{R}} e^{z-\alpha z} - e^{\epsilon-\alpha z} = \sup_{z\in\mathbb{R}} f(z) \tag{10}$$

where $f(z) = e^{z-\alpha z} - e^{\epsilon-\alpha z}$. We have

$$f'(z) = e^{z-\alpha z}(1-\alpha) - e^{\epsilon-\alpha z}(-\alpha) = e^{-\alpha z}(\alpha e^\epsilon - (\alpha-1)e^z) \tag{11}$$

Clearly $f'(z) = 0 \iff e^z = \frac{\alpha}{\alpha-1}e^\epsilon \iff z = \epsilon - log(1 - 1/\alpha)$. Thus

$$\begin{aligned} c &= f(\epsilon - log(1 - 1/\alpha)) && (12)\\ &= \left(\frac{\alpha}{\alpha-1}e^\epsilon\right)^{1-\alpha} - e^\epsilon\left(\frac{\alpha}{\alpha-1}e^\epsilon\right)^{-\alpha} && (13)\\ &= \left(\frac{\alpha}{\alpha-1}e^\epsilon - e^\epsilon\right)\left(\frac{\alpha}{\alpha-1}e^{-\epsilon}\right)^\alpha && (14)\\ &= \frac{e^\epsilon}{\alpha-1}\left(1 - \frac{1}{\alpha}\right)^\alpha e^{-\alpha\epsilon}. && (15) \end{aligned}$$

Thus

$$\mathbb{E}[max\{0, 1 - e^{\epsilon-Z}\}] \leq \frac{e^\epsilon}{\alpha-1}\left(1 - \frac{1}{\alpha}\right)^\alpha e^{-\alpha\epsilon}e^{(\alpha-1)\tau} = \frac{e^{(\alpha-1)(\tau-\epsilon)}}{\alpha-1}\left(1 - \frac{1}{\alpha}\right)^\alpha = \delta \tag{16}$$

□

Theorem 1.4. [ALC+21, Lemma 1, Equation (25)] Let M satisfy (α, γ)-RDP for $\alpha > 1$. If $\epsilon > \gamma$, then M is (ϵ, δ)-differentially private for

$$\delta = \frac{e^{(\alpha-1)\gamma} - 1}{\alpha\left(e^{(\alpha-1)\epsilon} - 1\right)}. \tag{17}$$

Proof. The second term of [ALC+21, Lemma 1, Equation (25)] states that, when $0 < \alpha\delta < 1$, the smallest guaranteed approximate-DP epsilon is at most

$$\frac{1}{\alpha-1}\log\left(\frac{e^{(\alpha-1)\gamma} - 1}{\alpha\delta} + 1\right). \tag{18}$$

Substitution of the stated δ makes this expression equal to ϵ. Moreover, $\epsilon > \gamma$ implies $e^{(\alpha-1)\epsilon} - 1 > e^{(\alpha-1)\gamma} - 1$, and therefore $0 < \alpha\delta < 1$, satisfying the lemma's condition. □

[1] This is the definition of (α, τ)-Rényi differential privacy.

Corollary 1. [CKS20] Let $M : \mathcal{X}^n \to \mathcal{Y}$ be a randomized algorithm. Let $\alpha \in (1, \infty)$ and $\epsilon \geq 0$. Suppose $D_\alpha(M(x)||M(x')) \leq \tau$ for all $x, x' \in \mathcal{X}^n$ differing in a single entry. Then M is (ϵ, δ)-differentially private for

$$\epsilon = \tau + \frac{\ln(1/\delta) + (\alpha - 1)\ln(1 - 1/\alpha) - \ln(\alpha)}{\alpha - 1} \tag{19}$$

Proof. This follows by rearranging 1.3. □

Corollary 2. Let $M : \mathcal{X}^n \to \mathcal{Y}$ be a randomized algorithm satisfying ρ-concentrated differential privacy. Then M is (ϵ, δ)-differentially private for any $0 < \delta \leq 1$ and

$$\epsilon = \inf_{\alpha \in (1,\infty)} \alpha\rho + \frac{\ln(1/\delta) + (\alpha - 1)\ln(1 - 1/\alpha) - \ln(\alpha)}{\alpha - 1} \tag{20}$$

Proof. This follows from 1 by taking the infimum over all divergence parameters α. □

2 Certified inverse

For a fixed order α, the expression in 1 is the inverse of the branch-zero log-delta bound. The directed interval implementation evaluates this expression with $\gamma = \alpha\rho$ and returns an upper endpoint, so that endpoint is a sufficient epsilon for the requested delta.

For the second branch, the theorem requires $\epsilon > \gamma$ and its inverse additionally requires $\alpha\delta < 1$. Solving the bound in 1.4 gives

$$\epsilon = \frac{1}{\alpha - 1}\,\mathrm{softplus}\left(\log(\exp((\alpha - 1)\gamma) - 1) - \log(\alpha) - \log(\delta)\right). \tag{21}$$

The implementation represents a branch that cannot establish these strict domains as ‘None‘; numerical failures remain errors. Thus every certified candidate is independently a valid upper bound, and the minimum of the candidates is valid as well.

Native floating-point arithmetic is used only to select promising orders. The final bracket endpoints and optimizer argument are recomputed with outward-rounded ‘DInterval‘ arithmetic. Search accuracy therefore affects only tightness, not soundness. The special cases $\rho = 0$, $\rho = \infty$, $\delta = 0$, and $\delta = 1$ return the corresponding trivial valid bounds before finite interval evaluation.

Theorem 2.1. For any valid ρ and δ, a successful return from `zcdp_epsilon` is a sufficient epsilon for the requested delta.

Proof. The fixed-order inverse formulas above are sufficient by their respective forward theorems. The optimizer only chooses the finite candidate orders; each candidate is certified independently with outward-rounded interval arithmetic. The upper endpoint is therefore conservative, and the minimum of independently valid bounds remains valid. The explicit special-case returns are valid by the trivial privacy bounds. Errors are propagated, so no uncertified arithmetic result is accepted. □

3 Hoare Triple

Precondition

Compiler-verified

None

Caller-verified

None

Pseudocode

```
def zcdp_delta(rho: float, epsilon: float) -> float:
    validate_nonnegative_finite_or_infinite(rho, epsilon)
    if rho == 0.0 or epsilon == infinity:
        return 0.0
    if rho == infinity:
        return 1.0

    # Ordinary floating-point optimization only selects candidate orders.
    # Every accepted candidate gives a valid bound.
    alpha_lower = 1.0 + sqrt(machine_epsilon)
    alpha_upper = choose_finite_search_bound(rho, epsilon)
    cks_bracket = minimize_in_log_domain(
        lambda alpha: approximate_cks_log_delta(alpha, rho, epsilon),
        alpha_lower,
        alpha_upper,
    )

    # Re-evaluate candidates with outward-rounded interval arithmetic.
    log_delta_upper = infinity
    for alpha in candidates(cks_bracket, alpha_lower, alpha_upper):
        alpha = interval_point(alpha)
        rho_interval = interval_point(rho)
        epsilon_interval = interval_point(epsilon)
        log_bound = (
            (alpha - 1) * (alpha * rho_interval - epsilon_interval)
            + alpha * log(1 - 1 / alpha)
            - log(alpha - 1)
        )
        log_delta_upper = min(log_delta_upper, log_bound.upper)

    # Asoodeh et al., Lemma 1, Equation (25), supplies a second bound
    # whenever epsilon > alpha * rho.
    if epsilon > rho:
        asoodeh_upper = interior_upper_bound(epsilon / rho)
        asoodeh_bracket = minimize_in_log_domain(
            lambda alpha: approximate_asoodeh_log_delta(
                alpha, rho, epsilon
            ),
            alpha_lower,
            asoodeh_upper,
        )
        for alpha in candidates(asoodeh_bracket):
            if epsilon <= alpha * rho:
                continue
            alpha = interval_point(alpha)
            x = (alpha - 1) * alpha * interval_point(rho)
            y = (alpha - 1) * interval_point(epsilon)
            log_bound = log_expm1(x) - log(alpha) - log_expm1(y)
            log_delta_upper = min(log_delta_upper, log_bound.upper)

    return conservative_exp_clamped_to_unit_interval(log_delta_upper)
```

Postcondition

Theorem 3.1. For any possible setting of ρ and ϵ, `zcdp_delta` either returns an error, or a δ such that any ρ-differentially private measurement is also (ϵ, δ)-differentially private.

Proof. The validation and special-case branches return an error or a trivial valid bound. Otherwise, ordinary floating-point optimization only selects a finite set A of candidate orders strictly greater than one. Its accuracy affects tightness but not soundness.

For each candidate, the code evaluates the logarithm of either the bound from 1.3, or, when $\epsilon > \alpha\rho$,

the bound from 1.4, with the zCDP guarantee $\gamma = \alpha\rho$. These evaluations use `DInterval`, whose operations round outwards, so each upper endpoint is at least the exact logarithm of the corresponding valid bound. The minimum remains valid because every candidate bounds the same privacy quantity from above.

Finally, directed interval exponentiation rounds upwards, including when the result is below the smallest positive `f64`; the result is clamped to the probability range. Any interval-evaluation failure is returned as an error rather than being used as a privacy result. Thus every successful returned `delta` satisfies the postcondition. □

References

[ALC+21] Shahab Asoodeh, Jiachun Liao, Flavio P. Calmon, Oliver Kosut, and Lalitha Sankar. Three variants of differential privacy: Lossless conversion and applications. *IEEE Journal on Selected Areas in Information Theory*, 2(1):208–222, 2021.

[CKS20] Clément L. Canonne, Gautam Kamath, and Thomas Steinke. The discrete gaussian for differential privacy. *CoRR*, abs/2004.00010, 2020.